Pools, Rules, and Tools: A Template-Integrated Resource Architecture

A Meta-Project Demonstrating Fonds, Rules, and Tools Integration in Research Software

Field	Value
Author	Daniel Ari Friedman[1]
Affiliation	[1] Active Inference Institute
Correspondence	daniel@activeinference.institute
ORCID	0000-0001-6232-9096
Version	1.0.0
Date	2026-07-10
License	CC-BY-4.0
Repository	Public template repository

Field	Value
DOI	10.5281/zenodo.21298888
Keywords	research software engineering, monorepo architecture, reproducibility, fonds, governance rules, tool discovery, graceful degradation

Author Contributions I

Daniel Ari Friedman: Conceptualisation, architecture design, module implementation (`fonds_reader`, `rules_applier`, `tools_invoker`, `integration`, `strong_rule_evaluator`, `figures`, `manuscript_variables`), manuscript writing (all sections), test suite design, exemplar resource authoring (fonds, rules, tools), validation.

Acknowledgements I

The author thanks the Active Inference Institute for hosting the public template repository and providing the infrastructure within which this exemplar was developed. The design of the three-layer resource architecture draws inspiration from the Unix philosophy (Raymond 2003) and the enterprise application patterns documented by (Fowler 2002).

Data Availability I

All source code, configuration files, and exemplar resources described in this paper are available in the public template repository under the `projects/templates/template_pools_rules_tools/` path. The integration pipeline is fully reproducible from source using `uv run python projects/templates/template_pools_rules_tools/scripts/02_run_integration.py` from the repository root. Generated manuscript variables are stored in `output/data/manuscript_variables.json` and injected at render time. Every figure in this manuscript, including the cover illustration, is likewise reproducible from source via `uv run python projects/templates/template_pools_rules_tools/scripts/05_generate_figures.py`, which writes current assets to `output/figures/` while preserving the checked-in `manuscript/figures/` sources.

Competing Interests I

The author declares no competing interests.

Reproducibility Checklist I

This exemplar targets full computational reproducibility for every quantitative claim and figure it makes:

Reproducibility Checklist I

Artefact	Regenerated by	Verified by
Integration counts (fonds/rules/tools)	`scripts/02_run_integration.py`	`tests/tools/test_integration.py` end-to-end assertion
Manuscript variable tokens	`scripts/03_generate_manuscript.py`	Rendered PDF contains no unresolved `{{...}}` tokens
All 9 figures (8 content + 1 cover)	`scripts/05_generate_figures.py`	`tests/figures/test_figures.py` (per-function file-existence checks)
$\geq$90% `src/` line coverage	`uv run pytest … --cov-fail-under=90`	CI + local pre-push gate
Combined PDF	full project pipeline, PDF rendering stage (4-pass xelatex + bibtex)	`pdftotext` scan for unresolved-reference markers and page-scale raster read

Reproducibility Checklist I

A reader who clones the repository and runs the listed commands in order reproduces every number and image in this document without manual intervention. Quantitative claims in this document — fond/rule/tool counts, project inventory counts, and the runtime bar-chart figures (fig. 5, fig. 7) — are derived from the integration result, declared figure contracts, or the project filesystem at render time. None of these values are hand-typed; negative controls in `tests/figures/test_manuscript_variables.py` change both integration inputs and a real temporary project tree to prove that token generation tracks its sources rather than emitting fixed constants.